Module 1: Introduction to SciPy

What is SciPy?

SciPy (Scientific Python) is an open-source Python library used for:

- Scientific Computing
- Data Analysis
- Machine Learning
- Statistics
- Signal Processing
- Image Processing
- Mathematical Optimization

SciPy is built on top of **NumPy**, so NumPy arrays are the primary data structure used.

Why Use SciPy?

Without SciPy:

- Complex mathematical calculations require hundreds of lines of code.
- Optimization problems are difficult.
- Statistical analysis is time-consuming.
- Numerical integration must be implemented manually.

With SciPy:

- Ready-made scientific functions
- Fast computation
- Accurate algorithms
- Industry-standard library

Installing SciPy

pip install scipy

Check version

```
import scipy

print(scipy.__version__)
```

SciPy Architecture

Python
  |
NumPy
  |
SciPy
  |

Statistics
Optimization
Interpolation
Integration
Linear Algebra
FFT
Signal Processing
Spatial Algorithms
Image Processing
Sparse Matrices

SciPy Modules

Module	Purpose
scipy.constants	Scientific constants
scipy.cluster	Clustering
scipy.datasets	Sample datasets

scipy.fft	Fast Fourier Transform
scipy.integrate	Numerical Integration
scipy.interpolate	Curve fitting
scipy.io	File operations
scipy.linalg	Linear Algebra
scipy.ndimage	Image Processing
scipy.optimize	Optimization
scipy.signal	Signal Processing
scipy.sparse	Sparse Matrix
scipy.spatial	Distance Algorithms
scipy.special	Mathematical Functions
scipy.stats	Statistics

---

1. scipy.constants

Contains predefined scientific constants.

Common Methods

Constant	Meaning
pi	π
golden	Golden Ratio
c	Speed of light
G	Gravitational Constant
h	Planck Constant
e	Electron Charge
Avogadro	Avogadro Number

Boltzmann Boltzmann Constant

Example

from scipy import constants

print(constants.pi)
print(constants.c)
print(constants.golden)

Output

3.1415926535
299792458.0
1.6180339887

Real-Life Example

GPS satellites use the speed of light constant to calculate exact distances.

---

2. scipy.linalg

Linear Algebra Module

Used in

- AI
- Data Science
- Robotics
- Finance

Matrix Creation

import numpy as np

A=np.array([[2,3],
        [5,7]])

Determinant

from scipy import linalg

print(linalg.det(A))

Output

-1

Real Life

Checking whether a system of equations has a unique solution.

Inverse Matrix

print(linalg.inv(A))

Used in

- Regression
- Neural Networks
- Image Processing

Eigen Values

values,vectors=linalg.eig(A)

print(values)

Applications

- Face Recognition
- PCA
- Google PageRank

- Recommendation Systems

Solving Linear Equations

2x+3y=8
5x+7y=19

A=np.array([[2,3],[5,7]])
B=np.array([8,19])

print(linalg.solve(A,B))

Output

[5 -0.666]

---

3. scipy.optimize

Optimization means finding the **best possible solution**.

Used in

- Delivery Routes
- Manufacturing
- AI
- Finance
- Portfolio Management

---

minimize()

Example

$f(x)=x^2+5$

from scipy.optimize import minimize

```
fun=lambda x:x**2+5

result=minimize(fun,0)

print(result.x)
```

Real-Life Example

A delivery company wants the shortest path to reduce fuel costs.

---

root()

Finds roots of equations.

```
from scipy.optimize import root

fun=lambda x:x+5

print(root(fun,0))
```

Real-Life Example

Finding break-even points in business.

---

curve_fit()

Used in Machine Learning.

```
import numpy as np
from scipy.optimize import curve_fit

def model(x,a,b):
   return a*x+b

x=np.array([1,2,3,4,5])
y=np.array([2,4,6,8,10])

params,_=curve_fit(model,x,y)
```

print(params)

Applications

- Sales Prediction
- Population Growth
- Demand Forecasting

---

4. scipy.integrate

Numerical Integration

Used for

- Area under curve
- Physics
- Machine Learning
- Finance

---

quad()

from scipy.integrate import quad

result,error=quad(lambda x:x**2,0,5)

print(result)

Output

41.66

Real-Life Example

Finding total electricity consumption.

---

dblquad()

Double Integration

```
from scipy.integrate import dblquad

result,error=dblquad(lambda x,y:x+y,
0,1,
lambda x:0,
lambda x:1)

print(result)
```

Applications

- Heat Distribution
- Surface Area

---

5. scipy.interpolate

Used to estimate missing values.

Methods

- interp1d
- CubicSpline
- griddata

---

interp1d()

```
from scipy.interpolate import interp1d

x=[1,2,3,4]
y=[10,20,30,40]

f=interp1d(x,y)

print(f(2.5))
```

Output

25

Real-Life Example

Weather prediction with missing sensor readings.

CubicSpline

from scipy.interpolate import CubicSpline

cs=CubicSpline(x,y)

print(cs(2.5))

Applications

- Road Design
- Medical Curves
- Stock Charts

---

6. scipy.stats

One of the most important modules in Data Science.

Contains

- Mean
- Median
- Mode
- Probability
- Hypothesis Testing
- Correlation
- Regression

Mean

from scipy import stats

data=[10,20,30,40]

print(stats.tmean(data))

Median

print(stats.scoreatpercentile(data,50))

Mode

print(stats.mode([1,2,2,3,3,3]))

Standard Deviation

import numpy as np

print(np.std(data))

Normal Distribution

from scipy.stats import norm

print(norm.cdf(1.96))

Applications

- IQ Scores
- Student Marks
- Manufacturing

---

t-test

from scipy.stats import ttest_ind

group1=[90,91,92,95]
group2=[85,84,86,83]

print(ttest_ind(group1,group2))

Real-Life Example

Compare two teaching methods.

Pearson Correlation

from scipy.stats import pearsonr

x=[1,2,3,4]
y=[10,20,30,40]

print(pearsonr(x,y))

Applications

- Stock Market
- Sales Analysis
- Marketing

---

7. scipy.signal

Signal Processing

Methods

- butter
- filtfilt

- find_peaks
- convolve

---

find_peaks()

from scipy.signal import find_peaks

x=[1,3,7,1,2,6,0,1]

peaks,_=find_peaks(x)

print(peaks)

Applications

- ECG Analysis
- Earthquake Detection
- Voice Recognition

---

8. scipy.fft

Fast Fourier Transform

Convert

Time Domain

↓

Frequency Domain

Example

from scipy.fft import fft

import numpy as np

x=np.array([1,2,3,4])

```
print(fft(x))
```

Applications

- Music Analysis
- Speech Recognition
- Image Compression

---

9. scipy.spatial

Distance Calculation

```
from scipy.spatial.distance import euclidean

print(euclidean([1,2],[3,4]))
```

Applications

- Face Recognition
- KNN Algorithm
- Delivery Distance

---

10. scipy.special

Special Mathematical Functions

Example

```
from scipy.special import factorial

print(factorial(5))
```

Output

```
120
```

Applications

- Probability
- Combinatorics
- Machine Learning

Real-Time Mini Project 1: Student Marks Analysis

Problem

Analyze students' marks to compute statistical measures and identify relationships between subjects.

Features

- Calculate mean, median, mode
- Compute standard deviation
- Find correlation between Math and Science scores
- Detect outliers using z-scores

Code

```
import numpy as np
from scipy import stats

math = np.array([75, 82, 90, 88, 95, 78, 84, 91, 87, 80])
science = np.array([72, 80, 89, 86, 96, 79, 82, 90, 85, 81])

print("Mean (Math):", stats.tmean(math))
print("Median (Math):", np.median(math))
print("Mode (Math):", stats.mode(math, keepdims=True))
print("Standard Deviation:", np.std(math))

corr, p = stats.pearsonr(math, science)
print("Correlation:", corr)

z_scores = stats.zscore(math)
```

```
outliers = math[np.abs(z_scores) > 2]
print("Outliers:", outliers)
```

Explanation

- tmean() computes the average.
- median() finds the middle value.
- mode() returns the most frequent value.
- pearsonr() measures the linear relationship between two subjects.
- zscore() helps detect unusually high or low marks.

Real-Time Mini Project 2: Sales Trend Forecast with Curve Fitting

Problem

Fit a linear model to monthly sales data and predict future sales.

Code

```
import numpy as np
from scipy.optimize import curve_fit

months = np.array([1, 2, 3, 4, 5, 6])
sales = np.array([120, 150, 170, 210, 230, 260])

def linear_model(x, a, b):
    return a * x + b

params, _ = curve_fit(linear_model, months, sales)

a, b = params
print("Slope:", a)
print("Intercept:", b)

future_month = 7
predicted_sales = linear_model(future_month, a, b)
print("Predicted Sales for Month 7:", predicted_sales)
```

Explanation

- curve_fit() estimates the best-fit line for the data.
- The fitted model is used to forecast the next month's sales.

Summary of Important SciPy Modules

Module	Purpose	Common Methods	Real-Life Applications
scipy.constants	Scientific constants	pi, c, G	Physics, Engineering
scipy.linalg	Linear algebra	det(), inv(), solve(), eig()	AI, Robotics, Finance
scipy.optimize	Optimization	minimize(), root(), curve_fit()	Route planning, Forecasting
scipy.integrate	Numerical integration	quad(), dblquad()	Energy, Finance
scipy.interpolate	Missing value estimation	interp1d(), CubicSpline()	Weather, Sensors
scipy.stats	Statistics	tmean(), ttest_ind(), pearsonr()	Data analysis, ML
scipy.signal	Signal processing	find_peaks(), butter()	ECG, Audio
scipy.fft	Frequency analysis	fft()	Audio, Images
scipy.spatial	Distance calculations	euclidean()	KNN, GIS
scipy.special	Special mathematical functions	factorial()	Probability, Statistics

Suggested Learning Path

To master SciPy for data science, study the modules in this order:

1. scipy.constants
2. scipy.linalg

3. scipy.stats
4. scipy.optimize
5. scipy.interpolate
6. scipy.integrate
7. scipy.spatial
8. scipy.signal
9. scipy.fft
10. scipy.special

Scikit-learn for Beginners (Data Science Using Python)

Scikit-learn is the **most popular Machine Learning library** in Python. It helps computers learn patterns from data and make predictions without being explicitly programmed.

What is Scikit-learn?

Imagine you are a teacher.

You show students many examples.

- Cat → Meow 🐱
- Dog → Bark 🐶
- Cow → Moo 🐄

After learning many examples, the student can identify a new animal.

Scikit-learn works in the same way.

It learns from **training data** and predicts **new data**.

Why Use Scikit-learn?

Suppose you own an online shopping website.

You want to answer questions like:

- Will a customer buy this product?
- Which customers may leave the service?
- What will next month's sales be?
- Which customers belong to the same buying group?

Scikit-learn helps answer all of these using Machine Learning.

Machine Learning Workflow

Collect Data
↓
Clean Data
↓
Feature Scaling
↓
Train Model
↓
Test Model
↓
Evaluate Model
↓
Predict New Data

Main Features of Scikit-learn

Feature	Purpose	Real-Life Example
Classification	Predict categories	Spam Email Detection
Regression	Predict numbers	House Price Prediction
Clustering	Group similar data	Customer Segmentation
Model Evaluation	Measure model performance	Check prediction accuracy
Feature Scaling	Normalize data	Prepare data before training

---

1. Classification

What is Classification?

Classification predicts **categories or labels**.

The output is **not a number**, but a class.

Example:

Input

Age = 24
Salary = 40000

↓

Output

Buy Product = YES

or

Email

↓

Spam
or
Not Spam

Real-Life Examples

Problem	Classes
Email Detection	Spam / Not Spam
Disease Detection	Sick / Healthy
Face Recognition	Person A / Person B

Loan Approval	Approved / Rejected
Student Result	Pass / Fail

Example Dataset

Study Hours	Attendance	Result
2	60	Fail
3	65	Fail
5	80	Pass
6	90	Pass
7	95	Pass

We train the computer.

Now give

Hours = 4
Attendance = 75

Prediction

Pass

Classification Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Predict
prediction = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, prediction))
```

Explanation

load_iris()

Loads the Iris flower dataset.

---

train_test_split()

Splits data into

- Training Data (80%)
- Testing Data (20%)

DecisionTreeClassifier()

Creates a Decision Tree classification model.

---

model.fit()

Learns from the training data.

---

model.predict()

Predicts flower categories for new data.

---

accuracy_score()

Calculates prediction accuracy.

Real-Time Project 1

Email Spam Detection

Input

Free Gift
Congratulations
Claim Prize

↓

Prediction

Spam

---

2. Regression

What is Regression?

Regression predicts **continuous numerical values**.

Examples:

House Price

₹35,00,000

Salary

₹60,000

Temperature

36°C

Real-Life Examples

Problem	Output
House Price	₹45 Lakhs
Stock Price	₹920
Temperature	30°C
Salary Prediction	₹70,000
Sales Prediction	₹2,50,000

Example Dataset

Experience	Salary
1	20000
2	25000
3	30000
4	35000
5	40000

Computer learns this relationship.

New Input

Experience = 6

Prediction

₹45000

Regression Program

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Input data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([20000, 25000, 30000, 35000, 40000])

# Create model
model = LinearRegression()

# Train
model.fit(X, y)

# Predict
salary = model.predict([[6]])

print("Predicted Salary:", salary)
```

Explanation

LinearRegression()

Creates a Linear Regression model.

---

fit()

Learns the relationship between experience and salary.

---

predict()

Predicts the salary for 6 years of experience.

Real-Time Project 2

House Price Prediction

Input

Area = 1500 sq.ft
Bedrooms = 3
Bathrooms = 2

↓

Prediction

₹48,50,000

---

3. Clustering

What is Clustering?

Clustering groups **similar data together**.

Unlike classification, clustering does **not** have predefined labels.

Imagine you have 1,000 customers.

You don't know who is rich or who spends more.

The computer automatically groups customers based on similar shopping behavior.

Real-Life Examples

Problem	Groups
Shopping	High, Medium, Low Spenders
College	Top, Average, Weak Students
Hospital	Similar Patients
Movies	Similar Viewers
Banking	Premium, Regular Customers

Customer Example

Customer	Spending
A	₹50,000
B	₹48,000
C	₹49,000
D	₹5,000
E	₹4,500

Computer creates

Cluster 1

A
B
C

Cluster 2

D
E

Clustering Program

```
from sklearn.cluster import KMeans
import numpy as np

data = np.array([
   [100],
   [120],
   [110],
   [500],
   [550],
   [530]
])

model = KMeans(n_clusters=2, random_state=42)

model.fit(data)

print(model.labels_)
```

Explanation

KMeans()

Creates a clustering model.

---

n_clusters=2

Makes two groups.

---

labels_

Shows which cluster each data point belongs to.

Example Output

[1 1 1 0 0 0]

The first three customers are in one cluster, and the last three are in another.

Real-Time Project 3

Customer Segmentation

Shopping Mall

Input

Age
Income
Shopping Score

↓

Output

Premium Customers

Regular Customers

Low Spending Customers

Marketing teams use these groups to send targeted offers.

---

4. Model Evaluation

What is Model Evaluation?

After training a model, we need to know **how well it performs**.

Model evaluation measures the quality of predictions.

Example:

Suppose a model predicts exam results for 100 students.

- Correct predictions = 95
- Wrong predictions = 5

Accuracy:

95%

Common Evaluation Metrics

Metric	Used For	Meaning
Accuracy	Classification	Percentage of correct predictions
Precision	Classification	Correct positive predictions
Recall	Classification	Ability to find all positive cases
F1 Score	Classification	Balance between precision and recall
MAE	Regression	Average absolute error
MSE	Regression	Average squared error
RMSE	Regression	Square root of MSE
R^2 Score	Regression	Goodness of fit

Accuracy Example

```
from sklearn.metrics import accuracy_score

actual = [1, 0, 1, 1, 0]
predicted = [1, 0, 1, 0, 0]

print("Accuracy:", accuracy_score(actual, predicted))
```

Output

Accuracy: 0.8

The model correctly predicted 4 out of 5 cases (80%).

Mean Absolute Error (MAE)

from sklearn.metrics import mean_absolute_error

actual = [100, 200, 300]
predicted = [110, 190, 295]

print(mean_absolute_error(actual, predicted))

The average prediction error is calculated, giving an idea of how far predictions are from actual values.

Real-Time Project 4

Loan Approval Prediction

The bank checks:

- Accuracy
- Precision
- Recall

before deploying the model to ensure reliable decisions.

---

5. Feature Scaling

What is Feature Scaling?

Feature scaling transforms features so they have similar ranges. This prevents variables with large values from dominating the learning process.

Example (Without Scaling)

Age	Salary

25	25,000
30	50,000
35	100,000

Here:

- Age ranges from 25 to 35.
- Salary ranges from 25,000 to 100,000.

Since salary values are much larger, many machine learning algorithms will give salary more influence than age.

Types of Feature Scaling

1. Standardization (StandardScaler)

Transforms data to have:

- Mean ≈ 0
- Standard deviation ≈ 1

Used by algorithms such as Logistic Regression, SVM, KNN, and PCA.

```
from sklearn.preprocessing import StandardScaler
import numpy as np

data = np.array([[25, 25000],
                 [30, 50000],
                 [35, 100000]])

scaler = StandardScaler()

scaled = scaler.fit_transform(data)

print(scaled)
```

Explanation

- fit() calculates the mean and standard deviation of each feature.

- transform() applies the scaling.
- fit_transform() performs both steps together.

---

2. Min-Max Scaling (MinMaxScaler)

Rescales features to a fixed range, usually **0 to 1**.

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

data = np.array([[25, 25000],
                 [30, 50000],
                 [35, 100000]])

scaler = MinMaxScaler()

scaled = scaler.fit_transform(data)

print(scaled)
```

Real-Time Project 5

Employee Salary Prediction

Input Features:

- Age
- Experience
- Salary
- Education

Before training the model:

1. Clean the data.
2. Scale numerical features using StandardScaler or MinMaxScaler.
3. Train the machine learning model.
4. Evaluate the model using appropriate metrics.
5. Use the trained model to predict salaries for new employees.

Summary

Concept	Purpose	Example	Common Scikit-learn Class/Function
Classification	Predict categories	Spam vs Not Spam	DecisionTreeClassifier, LogisticRegression
Regression	Predict numbers	House Price	LinearRegression
Clustering	Group similar records	Customer Segments	KMeans
Model Evaluation	Measure model quality	Accuracy, MAE	accuracy_score, mean_absolute_error
Feature Scaling	Normalize feature values	Age and Salary	StandardScaler, MinMaxScaler

real-time project with a graphical user interface (GUI) is one of the best ways to learn Scikit-learn.

I recommend building **two beginner-friendly projects** that cover the main supervised learning tasks:

1. **Classification Project:** Student Performance Prediction System
2. **Regression Project:** House Price Prediction System

Each project will include a complete UI and follow a real software development workflow.

Project 1: Student Performance Prediction (Classification)

Objective

Predict whether a student will **Pass** or **Fail** based on academic information.

Real-Time Scenario

A college wants to identify students who are at risk of failing before the final exam. Teachers can then provide additional support.

Technologies

- Python
- Scikit-learn
- Pandas
- NumPy
- Matplotlib
- Joblib
- Tkinter (Desktop GUI)

Project Structure

StudentPerformancePrediction/



├── dataset/

│ └── student.csv

├── models/

│ └── student_model.pkl

├── images/

├── train_model.py

├── app.py

├── utils.py

├── requirements.txt

└── README.md

Dataset

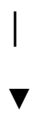
Study Hours	Attendance	Assignments	Internal Marks	Result
2	60	3	30	Fail
4	70	5	45	Pass
5	80	7	60	Pass
1	40	1	20	Fail

Machine Learning Workflow

Student Data



Data Cleaning



Feature Scaling

▼

Train Decision Tree Model

|

▼

Save Model (.pkl)

|

▼

Tkinter GUI

|

▼

Predict Pass / Fail

GUI Layout

```
+-----------------------------------------------+
|      Student Performance Prediction           |
+-----------------------------------------------+
| Study Hours        [____________]             |
| Attendance (%)     [____________]             |
| Assignments        [____________]             |
| Internal Marks     [____________]             |
|                                               |
|         [ Predict Result ]                    |
|                                               |
```

| Prediction : PASS |

| Accuracy : 96% |

+----------+

Features

- Input validation
- Prediction in one click
- Displays Pass/Fail result
- Model accuracy
- Easy-to-use interface
- Save prediction history (optional)

Project 2: House Price Prediction (Regression)

Objective

Predict the selling price of a house based on its features.

Real-Time Scenario

A real estate company wants to estimate house prices before listing properties.

Technologies

- Python
- Scikit-learn
- Pandas
- NumPy
- Joblib
- Tkinter

Dataset

Area (sq ft)	Bedrooms	Bathrooms	Parking	Price
1000	2	2	1	3500000
1200	3	2	1	4500000
1800	3	3	2	7000000
2200	4	3	2	9000000

Workflow

House Dataset



Clean Data



Feature Scaling



Linear Regression



GUI Layout

```
+-----------------------------------------------+
|            House Price Prediction            |
+-----------------------------------------------+
| Area (sq ft)      [_______________]           |
| Bedrooms          [_______________]           |
| Bathrooms         [_______________]           |
| Parking           [_______________]           |
|                                               |
|          [ Predict Price ]                    |
|                                               |
| Predicted Price                               |
| ₹ 72,50,000                                   |
```

+---+

Features

- Enter house details
- Predict selling price instantly
- User-friendly interface
- Error handling for invalid inputs
- Reusable trained model

Learning Concepts Covered

Topic	Student Project	House Project
Data Loading	✔	✔
Data Cleaning	✔	✔
Train-Test Split	✔	✔
Feature Scaling	✔	✔
Classification	✔	✘
Regression	✘	✔

Model Training	✔	✔
Prediction	✔	✔
Model Evaluation	✔	✔
Save Model (joblib)	✔	✔
Desktop GUI	✔	✔

Optional Enhancements

After completing the basic versions, you can extend the projects with:

- Login page
- SQLite/MySQL database
- User authentication
- Charts using Matplotlib
- Prediction history
- Export results to PDF or Excel
- Dark/Light theme
- Model retraining option
- CSV file upload for batch predictions
- Dashboard showing model statistics

Complete Source Code

These are substantial projects. Each one typically contains **1,000–2,000 lines of code** across multiple files (training scripts, GUI, utilities, dataset handling, and documentation), which is too large to provide in a single response.

I can generate them **step by step** with complete explanations, including:

1. **Dataset creation (CSV)**
2. **Model training (train_model.py)**
3. **Model evaluation and saving (joblib)**
4. **Professional Tkinter UI (app.py)**
5. **Input validation**
6. **Prediction logic**
7. **Error handling**
8. **Project report and documentation**

This approach ensures you understand every part of the project instead of just copying the code.

A good real-time project should include:

- Problem Statement
- Dataset
- Data Analysis
- Data Preprocessing
- Feature Scaling
- Model Training
- Model Evaluation
- Prediction
- Real-world Explanation

Below are **two complete beginner-level projects**.

Project 1: Student Pass/Fail Prediction (Classification)

Problem Statement

A college wants to predict whether a student will **Pass** or **Fail** before the final examination.

This helps teachers identify weak students and provide additional coaching.

Step 1: Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Create Dataset

```
data = {
   "StudyHours":[2,3,5,6,1,4,7,8,2,6],
   "Attendance":[60,65,80,90,50,75,95,98,55,88],
   "Assignments":[3,4,8,9,2,6,10,10,3,9],
   "Result":[0,0,1,1,0,1,1,1,0,1]
}

df = pd.DataFrame(data)

print(df)
```

Dataset

Study Hours	Attendance	Assignments	Result
2	60	3	Fail
3	65	4	Fail
5	80	8	Pass
6	90	9	Pass

Step 3: Select Features

```
X = df[["StudyHours","Attendance","Assignments"]]

y = df["Result"]
```

Explanation

Features (X)

Input columns

- Study Hours
- Attendance
- Assignments

Target (y)

Output

- Pass
- Fail

Step 4: Split Dataset

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Explanation

80% of the data is used for training, and 20% is reserved for testing.

Step 5: Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

Why Feature Scaling?

Suppose we have:

Study Hours	Attendance
2	90
8	95

Attendance values are much larger than study hours. Scaling converts them to a similar range so one feature does not dominate the model.

Step 6: Train Model

```
model = DecisionTreeClassifier()

model.fit(X_train,y_train)
```

The Decision Tree algorithm learns patterns from the training data.

Step 7: Prediction

```
prediction = model.predict(X_test)

print(prediction)
```

Example Output

[1 0]

- 1 = Pass
- 0 = Fail

Step 8: Accuracy

accuracy = accuracy_score(y_test,prediction)

print("Accuracy =",accuracy)

Example

Accuracy = 1.0

Step 9: Confusion Matrix

print(confusion_matrix(y_test,prediction))

Example

[[1 0]
 [0 1]]

Step 10: Classification Report

print(classification_report(y_test,prediction))

Displays:

- Precision
- Recall
- F1 Score
- Accuracy

Step 11: Predict New Student

```
new_student = scaler.transform([[5,85,8]])

result = model.predict(new_student)

if result[0] == 1:
   print("Student Will PASS")
else:
   print("Student Will FAIL")
```

Real-Time Workflow

Student Information

↓

Study Hours

↓

Attendance

↓

Assignments

↓

Machine Learning Model

↓

Prediction

↓

PASS / FAIL

Project 2: House Price Prediction (Regression)

Problem Statement

A real estate company wants to estimate the selling price of a house based on its characteristics.

Step 1: Import Libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Step 2: Dataset

```
data = {
   "Area":[1000,1200,1500,1800,2000,2200,2500,2700],
   "Bedrooms":[2,2,3,3,4,4,4,5],
   "Bathrooms":[1,2,2,3,3,3,4,4],
   "Price":[35,42,55,70,80,90,100,120]
}

df = pd.DataFrame(data)

print(df)
```

Step 3: Features

X = df[["Area","Bedrooms","Bathrooms"]]

y = df["Price"]

Step 4: Split Data

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=1
)
```

Step 5: Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

Step 6: Train Model

model = LinearRegression()

model.fit(X_train,y_train)

The model learns the relationship between house features and price.

Step 7: Predict

prediction = model.predict(X_test)

print(prediction)

Step 8: Evaluate

MAE

print(mean_absolute_error(y_test,prediction))

Measures the average absolute prediction error.

MSE

print(mean_squared_error(y_test,prediction))

Penalizes larger errors more heavily.

R^2 Score

print(r2_score(y_test,prediction))

Indicates how well the model explains the variation in house prices.

- 1.0 = Perfect fit
- 0.0 = Poor fit

Step 9: Predict New House

```
new_house = scaler.transform([[2100,4,3]])

price = model.predict(new_house)

print("Predicted Price =",price[0],"Lakhs")
```

Real-Time Workflow

House Details

↓

Area

↓

Bedrooms

↓

Bathrooms

↓

Machine Learning Model

↓

Predicted House Price

Concepts Learned in These Projects

Concept	Classification Project	Regression Project
Create Dataset	✔	✔

DataFrame	✔	✔
Feature Selection	✔	✔
Train/Test Split	✔	✔
Feature Scaling	✔	✔
Train Model	DecisionTreeClassifier	LinearRegression
Prediction	✔	✔
Accuracy	✔	—
Confusion Matrix	✔	—
Classification Report	✔	—
MAE	—	✔
MSE	—	✔
R^2 Score	—	✔
Predict New Data	✔	✔